

Diseño y Simulación de una ALU de 4 bits

LOPEZ HERNANDEZ JESUS FRANCISCO

Universidad Nacional Autónoma de México

Facultad de Estudios Superiores Cuautitlán

Diseño de Sistemas Digitales

México

Resumen—En esta práctica se diseñó una Unidad Aritmético-Lógica (ALU) de 4 bits utilizando circuitos combinacionales y el lenguaje SystemVerilog. La ALU permite realizar ocho operaciones diferentes mediante una entrada de control de 3 bits. Las operaciones implementadas fueron suma, resta, AND, OR, XOR, NOT y desplazamientos lógico a la izquierda y a la derecha.

Además del resultado de la operación, el sistema genera las banderas Carry, Overflow, Zero y Negative.

I. INTRODUCCIÓN

Una Unidad Aritmético-Lógica (ALU) es un circuito digital utilizado para realizar diferentes operaciones aritméticas y lógicas. Este tipo de unidad es una parte importante de los sistemas digitales y procesadores, ya que permite realizar operaciones sobre datos binarios.

En esta práctica se diseñó una ALU de 4 bits utilizando únicamente lógica combinacional. El sistema recibe dos entradas de datos denominadas *A* y *B*, ambas de 4 bits, y una entrada de control de 3 bits denominada *Operation*.

La entrada *Operation* permite seleccionar una de las ocho operaciones. Las operaciones implementadas se muestran en la Tabla I.

Cuadro I
OPERACIONES IMPLEMENTADAS EN LA ALU.

Operation	Operación
000	$A + B$
001	$A - B$
010	$A \& B$
011	$A B$
100	$A \wedge B$
101	$\sim A$
110	$A \ll 1$
111	$A \gg 1$

El sistema también cuenta con cuatro banderas que permiten conocer información adicional del resultado: *Carry*, *Overflow*, *Zero* y *Negative*.

II. DISEÑO GENERAL

El diseño se realizó mediante lógica combinacional, por lo que no requiere una señal de reloj para realizar las operaciones.

Las entradas y salidas principales del sistema son:

- *A*: entrada de datos de 4 bits.
- *B*: entrada de datos de 4 bits.
- *Operation*: selector de la operación de 3 bits.
- *Output*: resultado de la operación de 4 bits.
- *Carry*: indica el acarreo durante la suma o el borrow durante la resta.

- *Overflow*: indica un desbordamiento durante una operación aritmética.
- *Zero*: indica que el resultado obtenido es cero.
- *Negative*: indica que el bit más significativo del resultado es 1.

La selección de las operaciones se realizó mediante una estructura *case* dentro de un bloque *always_comb*. Esto permite que el valor de *Operation* determine qué operación debe realizar la ALU.

Para la suma se utilizó una variable temporal de 5 bits denominada *res*. El bit adicional permite conservar el acarreo generado al sumar dos valores de 4 bits.

III. FUNCIONES INTERNAS DE LA ALU

Cada combinación de la entrada *Operation* selecciona una función diferente. Para la suma se realiza $A + B$, mientras que para la resta se realiza $A - B$.

Las operaciones lógicas AND, OR y XOR se realizan bit a bit entre las entradas *A* y *B*. En la operación NOT únicamente se utiliza la entrada *A*, invirtiendo cada uno de sus bits.

Los desplazamientos también se realizan sobre la entrada *A*. En el desplazamiento a la izquierda los bits se mueven una posición hacia la izquierda, mientras que en el desplazamiento a la derecha se mueven una posición hacia la derecha.

Las banderas se calculan a partir del resultado obtenido. La bandera *Zero* se activa cuando el resultado es igual a cero y *Negative* toma el valor del bit más significativo del resultado.

IV. RESULTADOS DE LA SIMULACIÓN

Para comprobar el funcionamiento del sistema se realizó la simulación. Para las pruebas se utilizaron los siguientes valores:

$$A = 0101$$

$$B = 0011$$

La entrada *Operation* se modificó cada 10 ns para comprobar las ocho operaciones.

Los resultados esperados se muestran en la Tabla II.

Cuadro II
RESULTADOS DE LA SIMULACIÓN.

Tiempo	Operation	Operación	Output
0 ns	000	0101 + 0011	1000
10 ns	001	0101 − 0011	0010
20 ns	010	AND	0001
30 ns	011	OR	0111
40 ns	100	XOR	0110
50 ns	101	NOT A	1010
60 ns	110	A'1	1010
70 ns	111	A'1	0010

Los resultados obtenidos en la simulación coinciden con las operaciones esperadas. Para la suma se obtiene:

$$0101 + 0011 = 1000$$

mientras que para la resta:

$$0101 - 0011 = 0010$$

En las operaciones lógicas se observa que cada bit se procesa de acuerdo con la operación seleccionada.

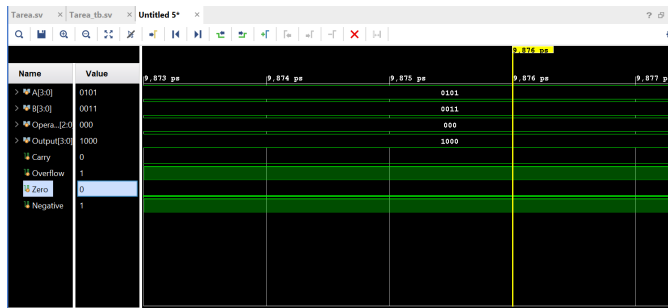


Figura 1. Resultado de la operación de suma.

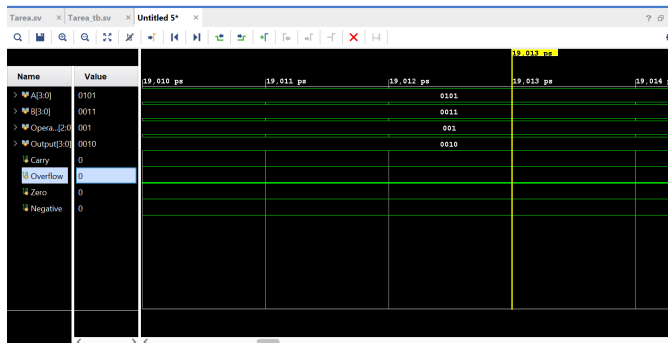


Figura 2. Resultado de la operación de resta.



Figura 3. Resultado de la operación AND.

La simulación permitió verificar que el selector *Operation* cambia correctamente entre las diferentes funciones y que las salidas correspondientes presentan los valores esperados.

V. CONCLUSIONES

Se logró diseñar una Unidad Aritmético-Lógica de 4 bits utilizando circuitos combinacionales y SystemVerilog.

La implementación permitió realizar las ocho operaciones solicitadas: suma, resta, AND, OR, XOR, NOT, desplazamiento lógico a la izquierda y desplazamiento lógico a la derecha.

Mediante la simulación en Vivado se comprobó el funcionamiento de las operaciones y de las banderas de salida. El uso de `always_comb` permitió describir el comportamiento combinacional del sistema, mientras que la estructura `case` facilitó la selección de las operaciones mediante la entrada *Operation*.

También durante una parte del código se sacaron fragmentos de apoyo, en este el `always_comb` y el `case` todo esto debido a que si ocupábamos solo `if` and `else` no me estaba quedando exactamente claro el funcionamiento sin embargo de esta manera fue mas claro observar el comportamiento de la ALU y que resultados esperar a parte de que a la hora de la descripción fue mucho mas simple.

REFERENCIAS

- [1] AMD, "Vivado Design Suite User Guide: Design Flows Overview," AMD.
- [2] IEEE, "IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language," IEEE Std. 1800.

APÉNDICE A CÓDIGO DE LA ALU

En este apéndice se presenta el código utilizado para implementar la ALU.

```
module Tarea (
    input logic [3:0] A,
    input logic [3:0] B,
    input logic [2:0] Operation,

    output logic [3:0] Output,
    output logic Carry,
    output logic Overflow,
    output logic Zero,
    output logic Negative
);

logic [4:0] temp;

always_comb begin

    Output = 4'b0000;
    Carry = 1'b0;
    Overflow = 1'b0;
```

```

temp = 5'b00000;

case (Operation)

    3'b000: begin

        temp = A + B;
        Output = temp[3:0];
        Carry = temp[4];

        Overflow = (~A[3] & ~B[3] & Output[3]) |
                    (A[3] & B[3] & ~Output[3]);

    end

    3'b001: begin

        Output = A - B;
        Carry = (A < B);

        Overflow = (A[3] & ~B[3] & ~Output[3]) |
                    (~A[3] & B[3] & Output[3]);

    end

    3'b010: begin
        Output = A & B;
    end

    3'b011: begin
        Output = A | B;
    end

    3'b100: begin
        Output = A ^ B;
    end

    3'b101: begin
        Output = ~A;
    end

    3'b110: begin
        Output = A << 1;
    end

    3'b111: begin
        Output = A >> 1;
    end

endcase

if (Output == 4'b0000)
    Zero = 1'b1;
else
    Zero = 1'b0;

Negative = Output[3];

end

endmodule

```

APÉNDICE B CÓDIGO DE SIMULACIÓN

```

`timescale 1ns / 1ps

module Tarea_tb;

    logic [3:0] A;
    logic [3:0] B;
    logic [2:0] Operation;

    logic [3:0] Output;
    logic Carry;
    logic Overflow;
    logic Zero;
    logic Negative;

    Tarea prueba (

```

```

        .A(A),
        .B(B),
        .Operation(Operation),
        .Output(Output),
        .Carry(Carry),
        .Overflow(Overflow),
        .Zero(Zero),
        .Negative(Negative)
    );

    initial begin

        A = 4'b0101;
        B = 4'b0011;

        Operation = 3'b000;
        #10;

        Operation = 3'b001;
        #10;

        Operation = 3'b010;
        #10;

        Operation = 3'b011;
        #10;

        Operation = 3'b100;
        #10;

        Operation = 3'b101;
        #10;

        Operation = 3'b110;
        #10;

        Operation = 3'b111;
        #10;

        $finish;

    end

endmodule

```

APÉNDICE C DIAGRAMA RTL

El diagrama RTL generado por Vivado se muestra en la Figura 4.

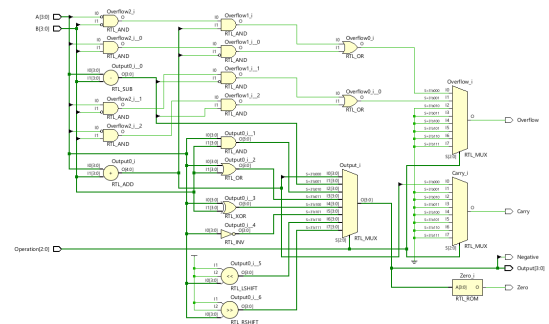


Figura 4. Diseño obtenido.